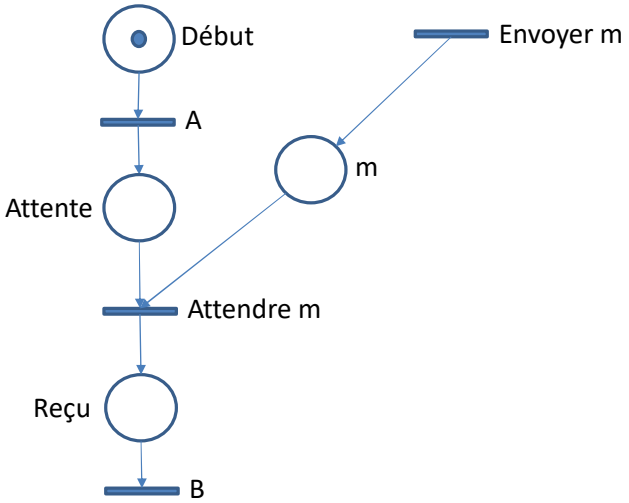
Détection de blocage

Processus Marius
Faire toujours
    Jouer une carte
    Ou
    Prendre le seau de glace
    Prendre boisson anisée et se servir

Processus Panis
Faire toujours
    Jouer une carte
    Ou
    Prendre boisson anisée
    Prendre le seau de glace et se servir

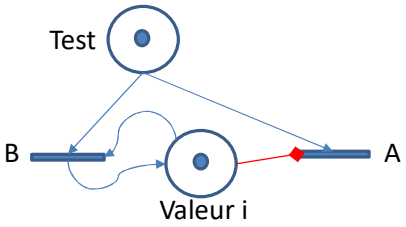
Sémantique d'un RdP

- Modéliser le programme suivant avec RdP
  - Faire action A
  - Attendre message m
  - Faire action B



Extension : arc inhibiteur

- Arc disant qu'une transition est franchissable si la place ne contient pas de jeton
- Augmente la puissance du RdP à celle d'une machine de Turing



Extension vitesse maximale

- Le franchissement des transitions devient déterministe, tout se passe comme si chaque transition avait une durée 1 et se devait d'être tirée en durée 1
- Augmente la puissance du RdP à celle d'une machine de Turing

Autres extensions

- RdP coloré
  - Couleurs de jetons, couleurs de valuations d'arcs
  - Permet de fusionner des places/transitions
- RdP à file
  - Les places contiennent des files FIFO de jetons, les arcs sont valués par des FIFO aussi
  - Modélisation d'échanges avec données ordonnées

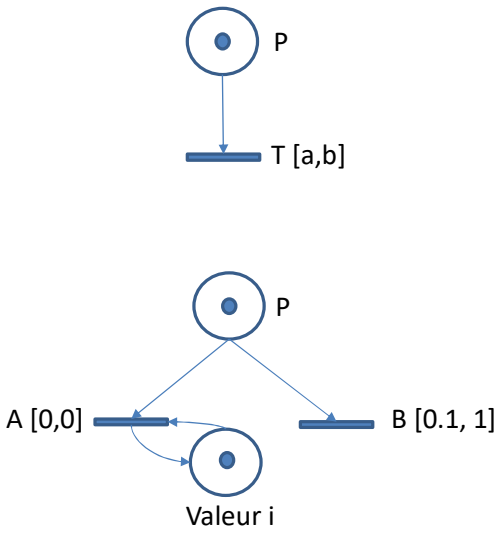
P. Merlin & D. Farber 1976

RÉSEAUX DE PETRI TEMPORELS
TIME PETRI NETS

Familles de RdP à temps

- Deux familles
  - RdP temporisés (timed Petri nets) : chaque transition possède une durée fixe – variantes où la place possède une durée de rétention du jeton
  - RdP temporels (time Petri nets) : les transitions possèdent un intervalle de temps de franchissement
    - ✓Très puissants pour modéliser l'incertitude du temps, par exemple durée d'exécution de tâche, durée de transmission, etc.

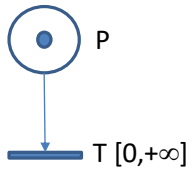
Principe



- A partir de l'instant t auquel la transition est franchissable au sens classique, elle devra être franchie entre les instants t+a et t+b inclus.
- Comme A est forcée de tirer dès qu'elle est franchissable mais que B doit attendre 0.1, on a ici un test à 0.

Formalisation

- Réseau de Petri : $N=(P,T,W,\alpha,\beta,M_0)$ où
  - P ensemble fini de places, $M_0 \in \mathbb{N}^{|P|}$ marquage initial
  - T ensemble fini de transitions $(P \cap T)=\varnothing$
  - $W: P\times T \cup T\times P \rightarrow \mathbb{N}$ fonction de valuation
    - ✓Pré $W_{|P\times T}$ fonction de précondition
    - ✓Post $W_{|T\times P}$ fonction de postcondition
  - Date de tir statique au plus tôt $\alpha: T \rightarrow \mathbb{Q}^+$
  - Date de tir statique au plus tard $\beta: T \rightarrow \mathbb{Q}^+ \cup \{+\infty\}$
    - ✓Intervalle de tir statique de t: $[\alpha(t), \beta(t)]$ avec $\alpha(t) \leq \beta(t)$



Sémantique RdP classique

Sémantique

- Transition $t \in T$ valide si $M \geq Pré(t)$
- Valide(M) est l'ensemble des transitions valides pour M
- Une transition t_k est **nouvellement** valide après le franchissement de t_i si ($t_k \notin Valide(M-Pre(t_i))$ et $t_k \in Valide(M-Pre(t_i)+Post(t_i))$) ou ($t_i=t_k$).
- Initialement, toute transition valide est nouvellement valide

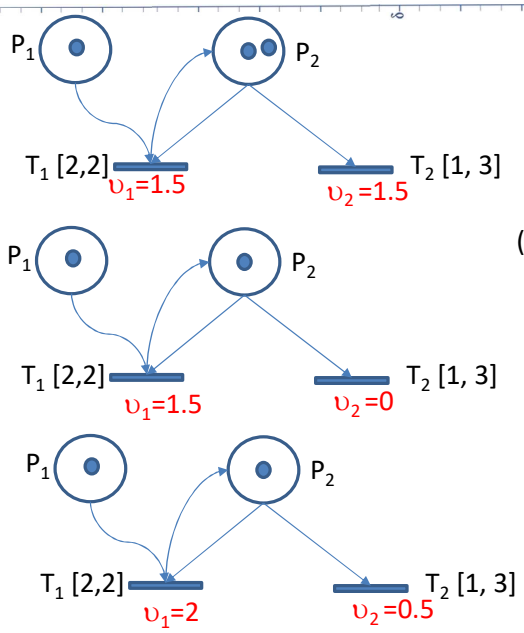
Sémantique

- L'état d'un RdP temporel N est définie par (M,Y)
  - $M \in \mathbb{N}^{|P|}$ marquage du RdP
  - $Y \in \mathbb{R}^{+|T|}$ horloges locales de transitions
- Une transition **continue** fait avancer les horloges d'un délai $d \in \mathbb{R}_{\geq 0}$ dans la limite de leur date de tir statique au plus tard β
  - $(M, Y) \xrightarrow{d} (M, Y+d)$ avec $\forall t \in$ Valide(M), $Y(t)+d \leq \beta(t)$
- Une transition **discrète** franchit une transition franchissable t (transition valide dont l'horloge locale a atteint sa date de tir statique au plus tôt) et réinitialise les horloges locales des transitions nouvellement valides
  - $(M, Y) \xrightarrow{t} (M', Y')$ avec $t \in$ Valide(M), M'=M-Pre(t)+Post(t), $Y(t) \geq \alpha(t)$, $\forall t'$ nouvellement valide, $Y'(t')=0$, pour les autres $Y'(t')=Y(t')$

Sématique

- Une transition est une séquence d'une transition continue suivie d'une transition discrète
  - $(M, Y) \xrightarrow{d} (M, Y+d) \xrightarrow{t} (M', Y')$
- Elle peut s'écrire
  - $(M, Y) \xrightarrow{d,t} (M', Y')$
- Une séquence de transitions est la séquence $(d_1,t_1)(d_2,t_2)$… de transitions $(M, Y) \xrightarrow{d1,t1} (M_1, Y_1) \xrightarrow{d2,t2} (M_2, Y_2)$…
- La séquence non temporisée est t_1t_2…

Exemple



(M,Y)=([1,2],[0,0])

Transition continue de d=1.5

(M,Y+1.5)=([1,2],[1.5,1.5])

Transition discrète de T_2: ([1,2],[1.5,1.5])(T2>([1,1],[1.5,0])

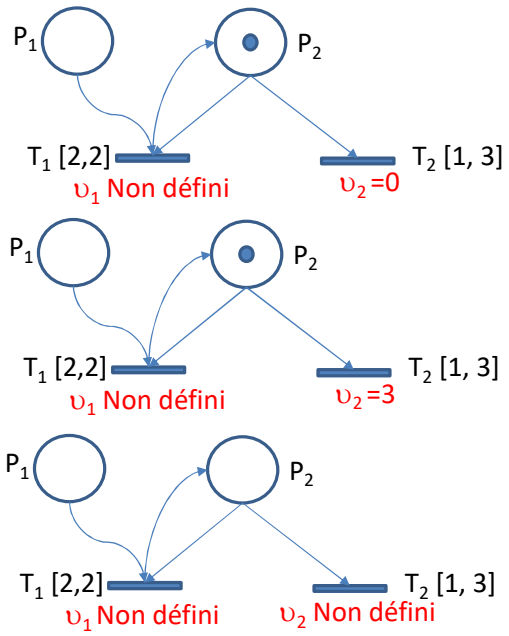
T_2 nouvellement franchissable

(M,Y')=([1,1],[1.5,0])

Transition continue de d=0.5

(M,Y'+0.5)=([1,1],[2,0.5])

Exemple/suite



Transition discrète de T_1: ([1,1],[2,0.5])(T_1>([0,1],[∅,0])

T_2 nouvellement franchissable

(M,Y'')=([0,1],[∅,0])

Il est inutile de tenir le compte

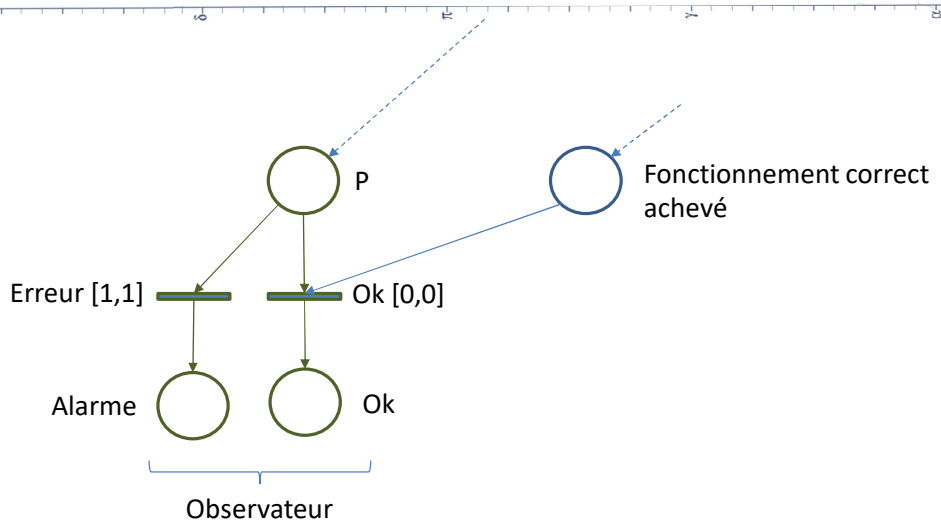
Transition continue de d=3

(M,Y''+3)=([0,1],[∅,3])

Transition discrète de T_2: ([0,1],[∅,3])(T2>([0,0],[∅, ∅])

(M,Y''')=([0,0],[∅,∅])

Modélisation de chiens de garde



Propriétés

- Marquage atteignable
- Pseudo-vivacité, quasi-vivacité, vivacité
- Logique temporelle: plusieurs logiques temporelles peuvent être étudiées sur un RdP temporel
  - LTL, CTL, CTL*

Linear Tree Logic LTL

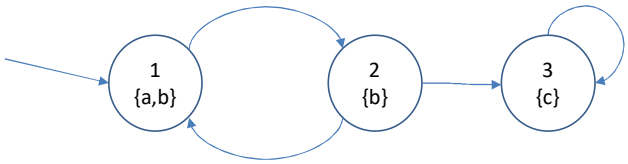
- Dans notre contexte, si l'on considère une séquence non temporisée de transitions $w=t_1t_2$… on peut vouloir vérifier des propriétés LTL (par exemple sur les marquages atteints sur une séquence possible):
  - neXt(f) noté Xf : f est vraie à la prochaine étape
  - Globally(f) noté Gf: f est vraie dans tous les états à partir de maintenant
  - Finally(f) noté Ff : f est vraie dans un état futur
  - f Until g noté fUg : f vraie jusqu'à ce que g soit vraie (g doit être vraie à un certain point)
  - f Weakuntil g noté fWg :comme U sauf que f peut être toujours vraie (auquel cas g n'est pas forcément vraie à un certain point)
    - ✓ $fWg \equiv (fUg) \vee Gf$

Structure de Kripke

$M = (S, I, L, \rightarrow)$

Système de transitions de Kripke, S ensemble d'états, $I \subseteq S$ états initiaux, $\rightarrow \subseteq S \times S$ transitions tel que tout état possède au moins un successeur, fonction d'interprétation $L: S \rightarrow 2^{PA}$ où PA ensemble de propositions atomiques

Deadlock représenté par transition en boucle sur le même état



$M = (S = \{1,2,3\}, I = \{1\}, L, \rightarrow)$
L(1)={a,b}, etc.
PA={a,b,c}

- b est-il vrai sur tout chemin à partir de 1 ?
  - $(M,1) \not\models AGb$ cependant $(M,1) \models AFb$
- b est-il vrai sur tout chemin jusqu'à ce que c soit vrai à partir de 1 ?
  - $(M,1) \models A(bWc)$
- Existe-t-il un chemin à partir de 3 tel que b soit vrai ?
  - $(M,3) \not\models EFb$

Computation Tree Logic (CTL et CTL*)

- S'applique sur un arbre de chemins possibles dans une structure de Kripke (le graphe des marquages d'un RdP ou le graphe de couverture peut être vu comme une telle structure, l'interprétation étant représentée par le marquage)
- Exprime des propriétés LTL avec quantificateur de chemin (∀ A All, ∃ E Exists)
  - Af : f vraie sur tout chemin (à partir de l'état courant)
  - Ef : f vraie sur au moins un chemin (à partir de l'état courant)
  - X, G, F, U, W : comme dans LTL
- Une structure de Kripke n'étant pas temporisée, ne considère pas le temps absolu, mais la causalité

Sémantique CTL*

- Formule d'état : $M = (S, I, L, \rightarrow)$ structure de Kripke, $s \in S$, Φ formule d'état. $(M,s) \models \Phi$ se lit l'état s satisfait la formule Φ
- $(M,s) \models \top$ $\quad (M,s) \not\models \bot$
- $(M,s) \models p \Leftrightarrow p \in L(s)$
- $(M,s) \models \neg\Phi \Leftrightarrow (M,s) \not\models \Phi$
- $(M,s) \models \Phi_1 \wedge \Phi_2 \Leftrightarrow ((M,s) \models \Phi_1) \wedge ((M,s) \models \Phi_2)$
- $(M,s) \models \Phi_1 \vee \Phi_2 \Leftrightarrow ((M,s) \models \Phi_1) \vee ((M,s) \models \Phi_2)$
- $(M,s) \models \Phi_1 \Rightarrow \Phi_2 \Leftrightarrow ((M,s) \not\models \Phi_1) \vee ((M,s) \models \Phi_2)$
- $(M,s) \models A\phi \Leftrightarrow$ (∀chemin σ partant de s, $\sigma \models \phi$)
  - ϕ formule de chemin, $\sigma = s_1 s_2 s_3 \ldots$ tel que $s_1 = s$, $s_1 \rightarrow s_2$, $s_2 \rightarrow s_3$, etc.
- $(M,s) \models E\phi \Leftrightarrow$ (∃ chemin σ partant de s, $\sigma \models \phi$)

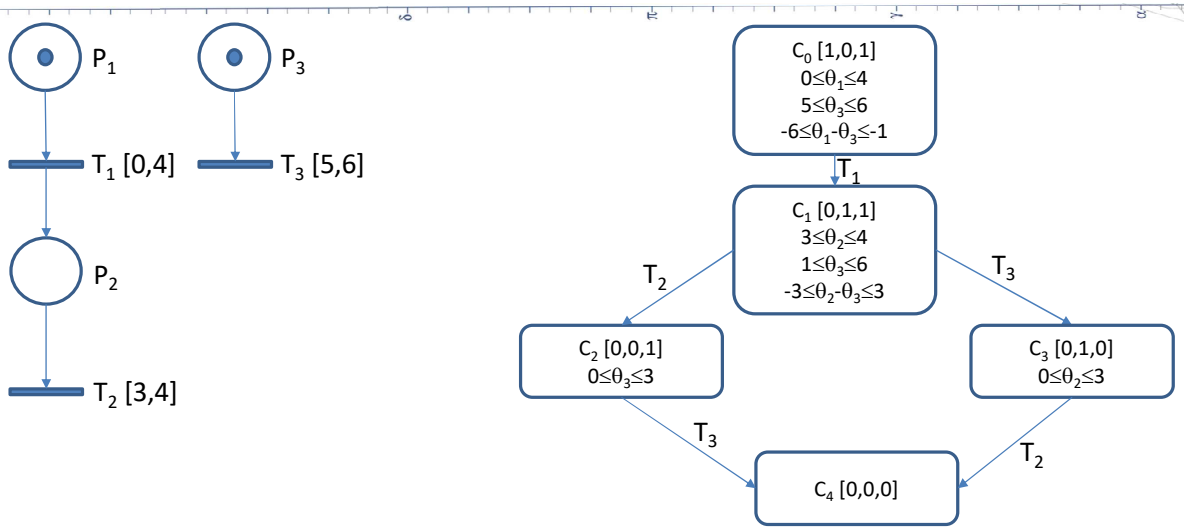
CTL* formules de chemin

- Soit un chemin $\sigma = s_0 \rightarrow s_1 \rightarrow \ldots$. On note le suffixe $\sigma(i)$ du chemin le chemin $s_i \rightarrow s_{i+1} \rightarrow \ldots$
- $\sigma \models X\phi \Leftrightarrow \sigma(1) \models \phi$
- $\sigma \models F\phi \Leftrightarrow (\exists i \geq 0 / \sigma(i) \models \phi)$
- $\sigma \models G\phi \Leftrightarrow (\forall i \geq 0 / \sigma(i) \models \phi)$
- $\sigma \models \phi_1 U \phi_2 \Leftrightarrow (\exists i \geq 0 / \sigma(i) \models \phi_2 \wedge \forall 0 \leq j < i, \sigma(j) \models \phi_1)$

Etude de comportement

- Le TPN s'exécute en temps continu, en théorie il y a donc une infinité d'états possibles
- Il faut regrouper des états « équivalents » de façon à obtenir un graphe fini si le marquage est fini
  - Graphe de classes
  - Graphe de régions géométriques
  - Graphe de classes fort
  - Graphe basé zones
  - Graphe de classes atomique
- Cependant, en fonction du regroupement un certain nombre de propriétés LTL ou CTL* ou temporisées sont perdues

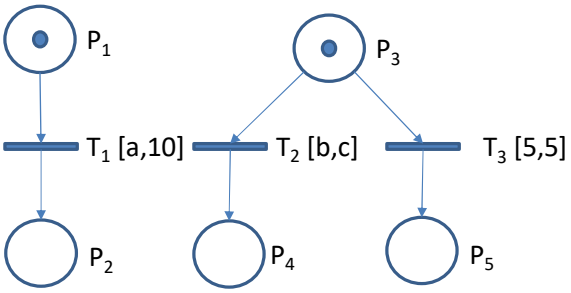
Graphe de classes



- Remarque: si θ_1 vaut 0, en réalité seule la séquence T_2,T_3 est permise, alors que si θ_1 vaut 4, seule T_3,T_2 est permise. Le graphe des classes ne le montre pas. Il conserve cependant les propriétés LTL et CTL* qui ne se basent pas sur du temps.

TPN paramétrique

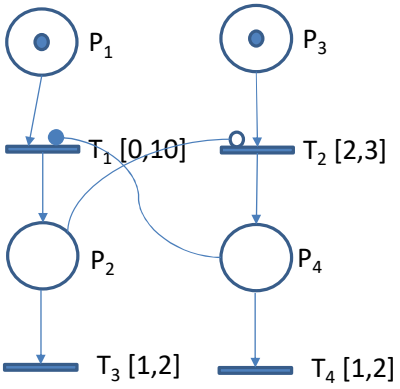
- On ajoute à la définition du TPN
  - Par={λ_1, λ_2,..., λ_l} ensemble de paramètres
  - $D_{Par} \subseteq \mathbb{Q}^{+Par}$ polyèdre convexe contraignant les paramètres



Par={a,b,c}
0≤a≤10
0≤b≤c

Arcs inhibiteurs et stopwatchs

- Inhibiteur logique : transition non valide si place amont marquée ●
- Inhibiteur stopwatch : transition non valide si place amont marquée, mais horloge locale conservée ○



Si T_1 franchie au bout d'une unité de temps, P_2 est marquée, et l'horloge locale de T_2 vaut 1, tout se passe donc comme si l'intervalle de franchissabilité de T_2 était [1,2]. Cette horloge est « gelée » par la stopwatch.
Si T_3 est franchie n'importe quand dans [1,2], à partir de cet instant, l'horloge de T_2 est « dégelée » et T_2 pourra être franchie entre 1 et 2 unités de temps plus tard. T_2 n'est pas nouvellement valide (horloge locale non réinitialisée)
Si T_2 est franchie en premier, T_1 n'est pas valide tant que P_4 est marquée. Au bout d'une à deux unités de temps, T_4 est franchie, T_1 est nouvellement valide.
On peut mettre des poids sur les inhibiteurs et stopwatchs, ils jouent alors leur rôle inhibiteur si la place amont est marquée par des jetons dont le nombre est au moins le poids.

Analysibilité

- De façon générale, l'accessibilité d'un TPN avec paramètres et inhibiteurs n'est pas décidable
- Si il n'existe que des paramètres en tant que borne inférieure ou supérieure d'intervalle de tir statique, et qu'aucun n'est à la fois borne inférieure et supérieure de deux intervalles différents, l'accessibilité est décidable.
- Construction d'un graphe de classe paramétrique sur un principe similaire au graphe de classes

TPN-TCTL

- Generalized Mutual Exclusion Constraint GMEC
- GMEC: GMEC and GMEC | GMEC or GMEC | GMEC => GMEC | not GMEC | $\sum_{i=1}^{m} a_i M(p_i)$ COMP c
  - $a_i \in \mathbb{Z}$, M marquage, p_i place du RdP
  - COMP$\in\{<,\leq,=,\geq,>\}$, c$\in\mathbb{N}$
- TPN-TCTL s'exprime sur une structure de Kripke (graphe des zones paramétriques) dans laquelle la fonction d'interprétation est une combinaison linéaire exprimée à base des marquages

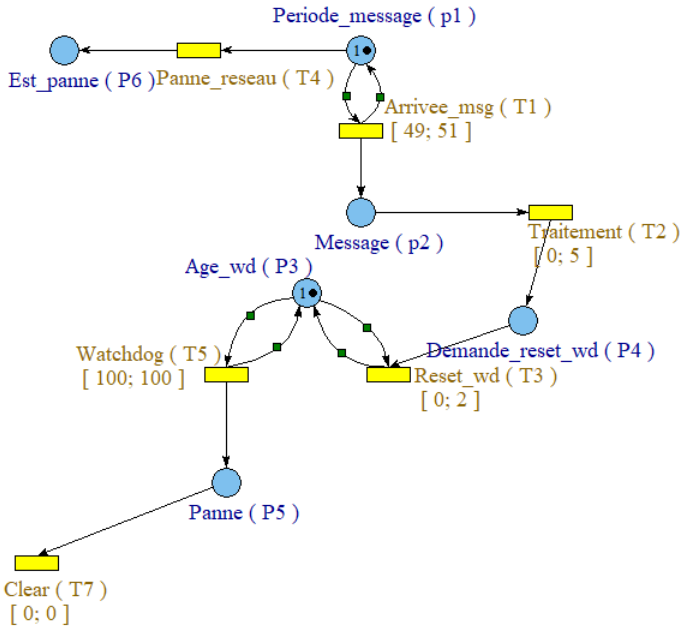
ParamTPN-PTCTL

- EφU[a,b]ψ
  - φ,ψ GMEC
  - a,b rationnels ou paramètres
  - Il existe un chemin possible dans lequel pour une date r∈[a,b] tel que ψ vraie et que pour toute date antérieure, φ vraie
- AφU[a,b]ψ
  - Pour tout chemin possible …
- EF[a,b]φ : il existe un chemin sur lequel on trouve une date dans [a,b] telle que φ vraie
- AF, EG, AG
- φ-->[a,b]ψ équivalent à AG(φ=> AF[a,b]ψ)

Exemple

- Un message est (normalement) reçu toutes les 50 ms, mais il y a une incertitude de 2% sur les horloges
- Il est traité en un temps maximal de 5 ms par le système, ce qui réinitialise le watchdog
- Un watchdog est déclenché lorsqu'il n'a pas été réinitialisé pendant 100 ms. Le traitement du watchdog prend au pire 2 ms.
- Quel est le delai maximal entre l'occurrence d'une panne et la fin du watchdog ?

Exemple dans Romeo



(P6==1)-->[0,a](P5==1)

a in [107, inf[

Pour garantir que P5 est marquée après que P6 soit marquée, il faut un intervalle d'au moins 107 u.t.

Conclusion

- Outil puissant de modélisation et d'analyse
- Plus le modèle est puissant, plus il est complexe à analyser
- Certaines analyses sont indécidables dans le cas général
- Le RdP temporel paramétrique à stopwatches peut être utilisé pour analyser finement les comportements possibles d'un système de tâches
  - Cependant, complexité élevée
  - Passage à l'échelle non assuré

TD Modélisation et analyse par Réseaux de Petri

Réseaux de Petri classiques

Exercice 1 Dîner des philosophes

Nous considérons 4 philosophes ayant un fonctionnement similaire : ils pensent ; lorsqu'ils ont faim ils commencent par prendre la baguette à leur gauche ; puis la baguette à leur droite ; puis ils mangent, déposent les baguettes et se remettent à penser. La Figure 1 représente un extrait du RdP à construire avec Roméo pour 4 philosophes et 4 baguettes. En supposant que la position des places baguette$_i$ est représentative des positions des baguettes sur la table, et que la position des transition modélisant le philosophe est aussi représentative de la position du philosophe à la table, nous voyons que le philosophe$_1$ qui fait face à la table prend la baguette à sa gauche avant de prendre la droite.

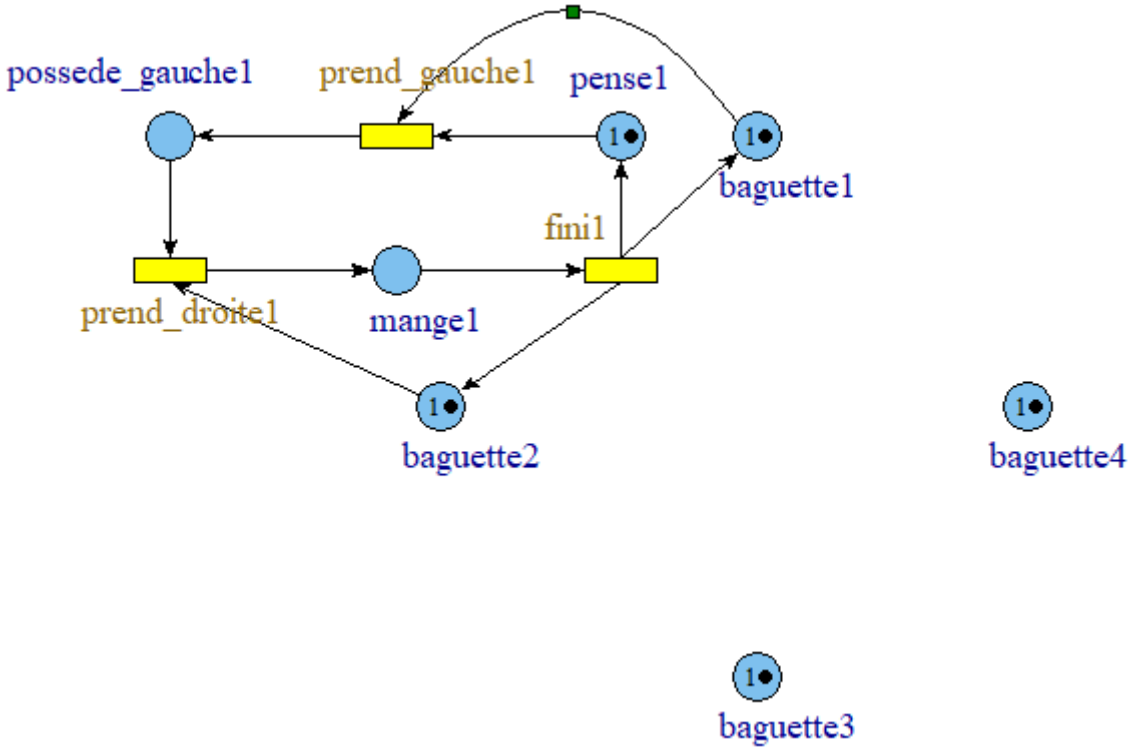


Figure 1 : extrait d'un RdP modélisant le dîner des philosophes à 4 philosophes et 4 baguettes

a) Compléter ce RdP, simuler quelques comportements.
b) Vérifier que le RdP est sauf (1-borné).
c) Tester s'il existe un deadlock.
d) Modifier l'ordre d'accès aux ressources afin qu'il ne soit possible de prendre les baguettes que dans l'ordre croissant de leur numéro. Vérifier à l'aide du *model checker* que le RdP est dorénavant sans deadlock.

Exercice 2 Producteur-consommateur

Producteur consommateur avec buffer en exclusion mutuelle

Lorsqu'il se présente, un producteur est prêt à produire une donnée dans un *buffer*. Déposer la donnée dans un buffer n'étant pas une action atomique, le *buffer* est protégé par un *mutex*, que le producteur va prendre avant de produire dans le *buffer*, et vendre (noter qu'en général dans ce cas, la production et la vente du mutex peuvent se représenter comme une action instantanée, puisque le producteur détient l'accès exclusif au *buffer*). Le consommateur fonctionne de façon duale, et, en assurant l'exclusion mutuelle, va consommer les données du *buffer*.

a) Modéliser ce comportement à l'aide d'un RdP.
b) Simuler le système et vérifier manuellement à l'aide de comportements que votre modèle fonctionne de façon attendue (par exemple respect de l'exclusion mutuelle lors de l'accès au *buffer*).

Attention, *checker* une propriété autre que la bornitude sur un RdP non borné plante en général le model checker de Romeo. D'une façon générale, il faut donc s'assurer de la bornitude d'un RdP avant de lancer une autre vérification de propriété. En cas de plantage du *model checker*, il faut tuer le processus romeo-cli. Sous windows, ce processus n'est pas vu par l'explorateur de processus, et un logiciel type ProcessExplorer sera alors utile pour tuer le processus sans avoir à redémarrer l'ordinateur. Romeo devra être relancé afin que les fichiers ouverts soient fermés et de nouveau accessibles pour le lancement d'une nouvelle analyse.

c) En observant le RdP (mais sans lancer l'analyse automatique) dire si le RdP est borné.

Version bornée du système

d) En s'inspirant de l'implémentation avec deux sémaphores à compte du paradigme du Producteur-Consommateur vu en cours de systèmes d'exploitation, proposer une modification du RdP telle que le *buffer* soit borné par 5.
e) Vérifier à l'aide d'une formule TCTL que le RdP est borné par 5.
f) S'assurer qu'il est possible que le *buffer* contienne 5 messages.

Exercice 3 Lecteur-écrivain

Nous voulons modéliser un système de salle d'examen contenant 5 places, dans laquelle des étudiants entrent et utilisent un poste de travail. Ils planchent individuellement sur le poste de travail, puis, lorsqu'ils ont terminé, vont poser leur devoir dans un *buffer*. Le *buffer* peut contenir jusqu'à 4 documents. Parallèlement à cela, un processus de correction automatique peut se lancer à tout moment : il va interdire l'accès au *buffer* lorsqu'il se lance, va corriger les documents un à un jusqu'à vider le *buffer*, puis autoriser à nouveau l'accès au buffer lorsqu'il a fini.

a) Modéliser ce comportement à l'aide d'un RdP et simuler manuellement quelques comportements.
b) Vérifier que le RdP est borné par 5.

Remarque : dans Romeo, il est pratique de nommer les places et transitions par un nom en rapport avec leur sémantique, mais cela est fort moins pratique pour faire référence aux places ainsi nommées lors de l'écriture d'une GMEC. En fait, les éléments nommés ont un label et un identificateur, l'identificateur étant le nom à utiliser dans une formule d'analyse. Il est donc conseillé de décocher l'option « identifier <- label » et de donner un label simple (type p1, p2, etc.) aux places utilisées dans des GMEC.

c) Vérifier que le *buffer* est borné par 4.

d) Vérifier que le *buffer* peut contenir jusqu'à 4 documents.
e) Vérifier que lorsqu'il y a correction, aucun étudiant n'a accès au *buffer* et vice-versa.

Réseaux de Petri temporels

Dans Romeo, cliquer sur le bouton Control Panel, et choisir l'extension « Stopwatch T-TPN ». Dans la partie « Parameters », choisir « Real parameters ».

Exercice 4 Dimensionnement de buffer

Nous considérons un *buffer* de sortie de switch avionique ARINC664p7 (AFDX™) que nous allons dimensionner en fonction des flux le traversant. Nous supposons que chaque trame de notre application traversant le réseau fait au plus 1kO. Les messages considérés sont sporadiques en sortie du *end-system* les émettant, avec un délai-minimal entre deux envois successifs appelé BAG (*Bandwidth Allocation Gap*). Cependant, dans une première étude nous allons les considérer périodiques.

Nous considérons quatre VL (*virtual links*) : un VL représente un chemin suivi par un message entre un nœud d'origine et un ou plusieurs nœuds de destination, à travers des switchs. Dans notre système, il n'y a qu'un switch, relié par un port bidirectionnel et des paires de câbles Ethernet dédiés 100 Mbps à chaque End/System. Nous considérons les VL 1 à 3, qui sont ceux qui empruntent en particulier la sortie du switch vers l'E/S4.

VL1 : E/S1 -> Switch -> E/S4

VL2 : E/S2 -> Switch -> E/S4

VL3 : E/S3 -> Switch -> E/S4

Le switch garantit qu'un message arrivant sur un port sera déposé à destination en 13µs au plus.

Id message	VL emprunté	BAG (ms)
1.1	1	1
1.2	1	2
1.3	1	4
2.1	2	4
3.1	3	8
4.1	3	1

a) Quelle est la durée d'émission d'une trame d'1kO sur un bus à 100 Mbps ?
b) Sachant que Romeo permet de n'exprimer les intervalles de tir statique qu'en nombres entiers, quelle unité choisir ?
c) Est-il nécessaire ici de représenter dans un modèle visant à dimensionner le *buffer* la vitesse de traversée d'un brin Ethernet ?
d) Modéliser le système par RdP temporel, en supposant que les messages sont émis périodiquement, sachant que nous nous intéressons à la taille nécessaire en sortie de switch.
e) Rechercher par dichotomie le marquage maximal de la place de sortie du switch, en déduire, en kO, la mémoire à prévoir dans ce buffer.
f) Les messages sont considérés sporadiques, modifier le RdP et tester la même propriété. Expliquer pourquoi le *model checker* ne répond pas.

Exercice 5 Ordonnancement à priorités fixes aux tâches de tâches indépendantes

Soit le système de trois tâches périodiques indépendantes à échéance sur requête données sous la forme $<C_i,T_i>$, avec τ_1 et τ_2 périodiques et τ_3 sporadique. $\tau_1<2,10>$, $\tau_3<3,10>$, $\tau_2<2,12>$. Nous choisirons

l'affectation de priorités RM qui est optimale dans ce contexte, et, afin de nous préparer à la suite et aux modifications de l'exercice qui arriveront par la suite, nous choisissons arbitrairement d'éviter les ex-aequo en affectant une priorité supérieure à τ_1.

a) Modéliser le système par un RdP temporel. Comme d'habitude, on s'assurera par simulation que le modèle semble se comporter correctement, et on vérifiera qu'il est borné avant de lancer d'autres analyses.
b) Calculer le pire temps de réponse de chaque tâche à l'aide d'une formule ParamTPN-PTCTL

Jusqu'ici pas vraiment d'intérêt à ce modèle, bien plus complexe à analyser qu'une RTA qui est dans ce cas exacte et nous donnerait le même résultat. Cependant, cela change avec l'adjonction de dépendances, telles que les ressources critiques ou les contraintes de précédence. Modifions notre modèle afin de représenter le système donné en exercice 4 du TD 5, et donné sur la Figure 2.

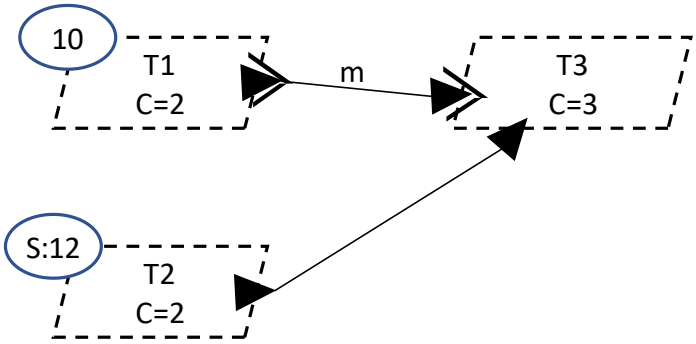


Figure 2 : Système de tâches avec exclusion mutuelle et précédence

Nous supposons que l'écriture (resp. lecture) dans un tableau noir s'opère en au plus 0,5 unité de temps en fin (resp. début) de tâche, alors que l'envoi (resp. réception) de message s'opère en au plus 0,6 unités de temps en fin (resp. début) de tâche. Nous supposons que le système est implémenté sur POSIX, avec l'utilisation du protocole de gestion de ressources à priorités plafonds (PCP) dans sa version immédiate.

c) Modifier le RdP précédent pour représenter les comportements possibles du système. Noter qu'il va nous falloir représenter des dixièmes d'unités de temps, Romeo n'autorisant que des valeurs entières pour les intervalles de tir statiques.
d) Calculer le temps de réponse de chaque tâche, ainsi que le temps de réponse de bout-en-bout du réveil de T1 à la terminaison de T3.

Exercice 6 Communication sans fil

Un drone doit utiliser une liaison radio pour envoyer des messages périodiques vers une station sol. La durée des messages dépend de leur taille, qui est variable en fonction des envois, et du débit de la liaison. Nous supposons l'existence de trois messages périodiques : m_1, de durée maximale 3 millisecondes (ms) et de période 10 ms ; m_2 de durée maximale 6 ms et de période 20 ms, et m_3 de durée maximale 15 ms et de période 30 ms.

a) Quelle est l'unité de temps à employer dans un réseau de Petri temporel ne manipulant que des entiers ? Justifier votre réponse.
b) Dans une première approche, nous supposons que chaque message est envoyé sur sa propre liaison radio, de façon à avoir un modèle simple à partir duquel nous construirons le reste. Proposer un réseau de Petri temporel modélisant l'envoi périodique des messages m_1 à m_3.

c) Afin d'éviter aux messages d'arriver simultanément, on laisse m_1 arriver dès le début, mais m_2 est décalé initialement de 3 ms avant d'être émis périodiquement aux dates 3 ms, 23 ms, 43 ms, etc. et m_3 est décalé initialement de 6 ms. Compléter le RdP précédent de façon à obtenir ce comportement.

d) En fait, il n'y a qu'une seule liaison radio, et les messages seront envoyés de façon non préemptive conservative. Cela signifie que s'il y a au moins un message à envoyer lorsque le canal est libre, un message est envoyé. Cependant nous n'avons pas connaissance de la stratégie employée pour choisir le message si plusieurs messages sont prêts lorsque le canal est libre. Proposer un RdP inspiré du RdP précédent permettant de prendre en compte cette politique. L'idée générale consiste à réserver la liaison dès que possible lorsqu'un message est prêt, et ne la libérer qu'à la fin de l'émission du message.

e) Exprimer en logique temporelle de Roméo comment calculer le pire temps de réponse du message m_1.

f) Exprimer en logique temporelle de Roméo comment calculer le pire temps de réponse du message m_3.

Nom :
Prénom :

Nom :
Prénom :

Nom :
Prénom :

Nom :
Prénom :